Ders 24 Genişletilmiş Notu: Sepet Analizi, Makrolar, Stringler, Sezar Şifreleme ve Görüntüde Ortalama Filtresi

Ders 24 Genişletilmiş Notu: Sepet Analizi, Makrolar, Stringler, Sezar Şifreleme ve Görüntüde Ortalama Filtresi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Sepet Analizi: Birliktelik Fikri
- 2. Sepet Analizinin Kodlanması
- 3. Makrolar
- 4. Stringler
- 5. Şifreleme: Sezar, Afin, Tablo ve Harf Sıklığı
- 6. Metinde Birlikte Geçen Kelimeler
- 7. Görüntüde Gürültü Temizleme: Ortalama Filtresi
- 8. Düzgün Bir Rapor Nasıl Hazırlanır?
- 9. Çok Büyük Sayılarla Çalışan Algoritmaları Denemek

1. Sepet Analizi: Birliktelik Fikri

Hoca, kalan derslerde işlenecek konuların ileride çok işe yarayacağını söyleyerek başladı. Bu haftanın ilk konusu sepet analizidir. Sepet analizi, bir kuruluşun müşterilerinin fişlerini veya alışveriş sepetlerini incelemek için kullanılan genel bir kavramdır. Veri madenciliği konusu olarak da geçer. Hoca burada anlatacağı algoritmanın çok verimli olmadığını, veri madenciliğinde daha iyi algoritmalar bulunduğunu, amacın yalnızca işin mantığını göstermek olduğunu belirtti.

Her ürünün bir ID'si vardır. Bu ID belirli bir ürünü veya bir ürün grubunu gösterebilir. Hocanın örneği kağıt mendildir: A markası veya B markası diye ayırmadan "kağıt mendil" tek bir ürün grubu sayılabilir. Sepette üç farklı markadan mendil olsa bile "burada bir tane kağıt mendil var" deriz. Sepetlerin de kendi ID'leri olur. Amaç, n müşterinin sepetlerini inceleyip bu inceleme sonucunda ilginç ilişkiler çıkarmaktır.

1.1 Birliktelik Analizi

Bu ilginç ilişkilere birliktelik analizi (association analysis) denir. Bir yerden kitap aldığınızda sepetin altında "bunu satın alanlar bunu da aldı" türünden öneriler görürsünüz. Sistem size sepetinizde olmayan ama sepetinizdeki ürünlerden bazılarına yakın ürünler gösterir. Bu yakınlık bazen makul, bazen de tamamen alakasız görünür.

> **Hocanın vurgusu:** Alışveriş yaparken gördüğünüz yazılımları mesleğinizle ilişkilendirmeye çalışın. Bir şey gördüğünüzde "burada şu tarz bir şey çalışıyor olabilir" diye düşünmek size çok şey kazandırır.

Makul birlikteliklere örnek olarak hoca şunları verdi: Patates alana soğan önerilebilir, fizik kitabı alana matematik kitabı önerilebilir. Bunlar birbiriyle ilgili olabilecek ürünlerdir.

Asıl ilginç olan, birbiriyle alakasız görünen ürünlerdir. Hoca, patates alanlara çivi veya diş macunu önerildiğini düşünmemizi istedi. Sepetler incelendiğinde örneğin "patates alanların %98'i çivi de almış" gibi bir sonuç çıkabilir. Bu belirli bir yöreye veya belirli bir markete özgü olabilir.

Bu ilişki korelasyondur, nedensellik değildir. Patates alan herkesin çivi alacağı anlamına gelmez, çivi almak patatese bağlı değildir. Nedensellik başka bir şeydir ve ikisini karıştırmamak gerekir.

> **Ek bilgi (derste anlatılmadı):** "Patates alanların %98'i" biçimindeki oran, birliktelik analizinde büyük olasılıkla güven (confidence) adıyla karşınıza çıkacaktır. Bir ürün çiftinin bütün sepetler içinde ne kadar sık görüldüğü de destek (support) adını alır. Bu derste yalnızca birlikte görülme sayısı hesaplanmaktadır.

Hoca, bu fikrin ABD'de yıllar önce yapılan bir çalışmayla ortaya çıktığını anlattı. Çalışmada bir bölgede bebek bezi alanların %90'ının başka bir ürünü de aldığı görülmüştü. Bu ürün bebek kremi veya bebek şampuanı gibi makul karşılanacak bir şey değildi, iki ürün arasında görünürde bir ilgi yoktu.

> **Ders sonrası not:** Altyazıda bozuk geçen ikinci ürün, bu anlatımla bilinen bebek bezi örneği düşünüldüğünde büyük olasılıkla biradır.

1.2 İçerik Tabanlı Öneriden Farkı

Sepete bir ürün attığınızda çıkan öneriler iki gruba ayrılabilir. Alakalı görünen öneriler genellikle ürünlerin özelliklerinin benzerliğine dayanır. Hoca kendi doktora tezini örnek verdi: Turkcell ile birlikte yürüttükleri bir projede, dinlediğiniz müziğin akustik özelliklerine en çok benzeyen 5 veya 10 müzik öneriliyordu. Bu, birbirine yakın karakteristikteki müzikler arasında yapılan içerik tabanlı (content mining) bir analizdir.

Sepet analizi ise tamamen sepetlere, yani insanların yönelimlerine ve eğilimlerine göre değişir. Alakasız görünen önerileri veren de bu analizdir. Bu derste işin temellerine bakılacaktır.

2. Sepet Analizinin Kodlanması

2.1 Veri Yapısı: Neden Düzensiz Liste Değil?

Hoca önce sepetlerin nasıl saklanacağını sordu. Öğrencilerden gelen ilk öneri bir matristi: satırlar müşterileri, sütunlar aldıkları ürünleri göstersin. Hoca bu önerinin mantıklı olduğunu ama bir sorun taşıdığını gösterdi. A müşterisi 1, 2, 3, 5, 8, 7 numaralı ürünleri, B müşterisi 3 ve 8'i, C müşterisi 0, 1 ve 5'i almış olsun. Satır sayısını müşteri sayısına göre ayarlayabiliriz, ama sütun sayısı satırdan satıra değişir. Bu tür dizilere düzensiz veya tırtıklı (ragged, jagged) dizi denir. Hem burada istenen bir yapı değildir, hem de henüz onu kodlayacak programlama bilgisine sahip değiliz.

Bunun yerine çok daha kolay bir yapı kullanılır. Her satırda bir sepet ID (müşteri ID de denebilir) ve bir ürün ID bulunur: "1 numaralı müşteri 2'yi aldı", "1 numaralı müşteri 3'ü aldı" gibi. Böylece sütun sayısı hep ikidir. Derste kullanılan başlangıç verisi şuydu:

```
int sepet[][2] = {{1,2}, {2,1}, {2,4}, {2,2}, {1,4}, {3,3}};
```

Kayıtlar karışık olabilir, sırası önemli değildir. Örneğin ilk kayıt "birinci sepette 2 numaralı ürün var", ikinci kayıt "ikinci sepette 1 numaralı ürün var" demektir.

2.2 Sepet-Ürün Matrisi A

Bu listeden kim neyi almış doğrudan ve sistemli biçimde görülemez. Listeyi müşteri numarasına göre sıralamak da istenen şeyi tam vermez. Çünkü asıl dert müşteriler değil, alınan ürünlerdir: "2 ile beraber en çok ne alınmış?" sorusuna cevap arıyoruz.

Bir öğrencinin önerdiği 3'e 4 matris fikrinden yola çıkan hoca şu dönüşümü istedi: Her sepet için her ürünün o sepette olup olmadığını gösteren bir matris kuralım. Ürün sepette varsa ilgili yere 1, yoksa 0 yazılır. Slayttaki sözde kodda matrisin satırları ürünleri, sütunları sepetleri gösteriyordu. Derste yazılan kodda ise satırlar sepet, sütunlar üründür.

Matrisin boyutları sepetlerin ve ürünlerin en büyük ID'sine göre bulunabilir ama hoca bunları kodda sabit verdi: ürün sayısı 5, sepet sayısı 4. ID'ler 1'den başladığı için 0 numaralı satır ve sütun hep boş kalır. Ayırılan yer için `#define MAX 10` ile 10×10'luk matris tanımlandı.

Matrisin başta sıfırlarla dolu olması gerekir. Hoca bunun için iki yol andı. Başına `static` yazmak matrisi sıfırlar. Ancak `static` başka amaçlarla da kullanılan bir anahtar kelimedir. İkinci yol ilk değer vermektir: `int A[MAX][MAX] = {{0}};` yazmak yeterlidir.

> **Hocanın vurgusu:** `static`'i sıfırlama amacıyla kullanmanın sakıncası yok ama bilinçli kullanmak lazım. Bilmediğiniz bir özelliği kullanınca "hocam sonuç yanlış geliyor" denen durumlar çıkıyor. Bu derste anlatılmadığı için şu an için kullanmamak en iyisidir.

Soru: İlk değer olarak yalnızca bir tane 0 yazsak da olur mu? **Cevap:** Olur, bir tane değer yazmak bile yeterlidir.

Matrisi görmek için bir yazdırma fonksiyonu yazıldı. Fonksiyon matrisi ve satır, sütun sayısını parametre olarak alır. Yerelde tanımlı `m` ile karışmaması için matrisin adı büyük `M` yapıldı.

A matrisini doldurmak için kaç döngü gerekir? Bir öğrenci iki `for` önerdi. Hoca, ikinci döngünün yalnızca iki sütunu gezmek için olacağını, oysa bu iki sütuna doğrudan erişebileceğimizi söyledi. Listedeki her kayıt için tek döngü yeterlidir:

```
for (i = 0; i < n; i++)
    A[sepet[i][0]][sepet[i][1]] = 1;
```

Bu satır şunu der: i. kaydın sıfırıncı sütunundaki sepet numarası ile birinci sütunundaki ürün numarasının kesiştiği yere git, orayı 1 yap. İlk denemede sütun sayısı eksik olduğu için son sütun görünmedi. Hoca kayıt sayısını $n = 6$, ürün sayısını 5 yapınca bütün kayıtlar göründü. Altı kayıt için sonuç şudur:

Sepet \ Ürün	0	1	2	3	4
0	0	0	0	0	0
1	0	0	1	0	1
2	0	1	1	0	1
3	0	0	0	1	0

Hoca sonucu kayıtlarla tek tek karşılaştırdı: $(1,2)$, $(2,1)$, $(2,4)$, $(2,2)$, $(1,4)$ ve $(3,3)$ konumlarında 1 vardır.

Soru: `i++` ile `++i` arasındaki fark nedir? **Cevap:** Birincisinde önce değer kullanılır, sonra artırılır. İkincisinde önce artırılır, sonra kullanılır.

> **Hocanın vurgusu:** Bu farkla ilgili sınavda da bir soru vardı.

2.3 Birlikte Satış Matrisi T

A matrisi hangi sepette hangi ürünün olduğunu söyler. Şimdi ürünler arasındaki ilişkiye bakacağız. Örneğin birinci sepette 2 ile beraber 4, ikinci sepette 2 ile beraber 3 alınmışsa, tek tek bakınca birliktelik kuramayız. Ama iki ayrı sepette 2 ile 4 birlikte alınmışsa, "2 ile 4, 2 defa birlikte alınmış" deriz. Yani i. ürünle j. ürünün kaç kez birlikte alındığının sıklığına ihtiyacımız var.

Bunun için ürün sayısı $\times$ ürün sayısı boyutunda, başta sıfırlarla dolu bir T matrisi kurulur. $T[i][j]$, i. ürünle j. ürünün kaç sepette birlikte bulunduğunu gösterir. Bir ürünün kendisiyle birlikte satışı anlamlı değildir, bu yüzden köşegen sıfır kalır. Sıra da önemli değildir: 1 ile 2 aynı sepetteyse 2 ile 1 de aynı sepettedir, tekrar bakmaya gerek yoktur. Bu nedenle matrisin yalnızca üst üçgeni doldurulur. Hoca, istenirse simetrik de yapılabileceğini belirtti.

Algoritmanın adımları şöyledir:

1. i birinci ürün olsun, 0'dan ürün sayısına kadar gitsin.
2. j ikinci ürün olsun, $i + 1$'den ürün sayısına kadar gitsin.
3. k sepet olsun, her i ve j için bütün sepetleri gezsin.

4. k. sepette hem i hem j varsa $T[i][j]$'yi 1 artır.

Hoca kodu yazarken iki hata yaptı ve bunlar öğretici oldu. İlki, bir öğrencinin uyarısıyla düzeltilen ve `k` yerine `j` yazılan döngüdür. İkincisi, koşulda indislerin ters yazılmasıdır. Slayttaki A matrisi ürün $\times$ sepet olduğu için `A[i][k]` yazılmıştı. Oysa kodumuzda A sepet $\times$ ürün olduğundan `A[k][i]` olmalıdır. Düzeltilmiş bütün program şudur (`gcc -Wall` ile uyarısız derlendi):

```c
#include <stdio.h>
#define MAX 10

void matrisYaz(int M[MAX][MAX], int m, int n)
{
    int i, j;
    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++)
            printf("%3d", M[i][j]);
        printf("\n");
    }
}

int main(void)
{
    int urunSay = 5, sepetSay = 4, n = 7;
    int sepet[][2] = {{1,2}, {2,1}, {2,4}, {2,2}, {1,4}, {3,3}, {3,2}};
    int A[MAX][MAX] = {{0}};
    int T[MAX][MAX] = {{0}};
    int i, j, k;

    for (i = 0; i < n; i++)
        A[sepet[i][0]][sepet[i][1]] = 1;
    printf("Sepet-urun matrisi A:\n");
    matrisYaz(A, sepetSay, urunSay);

    for (i = 0; i < urunSay; i++)
        for (j = i + 1; j < urunSay; j++)
            for (k = 0; k < sepetSay; k++)
                if (A[k][i] == 1 && A[k][j] == 1)
                    T[i][j]++;
    printf("Urunlerin birlikte satis sayilari T:\n");
    matrisYaz(T, urunSay, urunSay);
    return 0;
}
```

Hoca ilk çıktıyı görünce önce şüphelendi. 1 ile 2 birlikte bir kez satılmış, 1 ile 4 birlikte bir kez satılmış. Hatta "2 ile 4 neden 2 çıktı?" diye sordu. Sonra listeye değil, sepetlere bakmak gerektiğini gösterdi. Altı kayıtlı veriyle $T[2][4]$ adım adım şöyle bulunur:

- $k = 0$: $A[0][2] = 0$, sayılmaz.
- $k = 1$: $A[1][2] = 1$ ve $A[1][4] = 1$, sayaç 1 olur.
- $k = 2$: $A[2][2] = 1$ ve $A[2][4] = 1$, sayaç 2 olur.
- $k = 3$: $A[3][2] = 0$, sayılmaz.

Yani $T[2][4] = 2$ doğrudur. Ayrıca $T[1][2] = 1$ ve $T[1][4] = 1$ çıkar. Listedeki $(2, 2)$ kaydı "2 ile 2 birlikte satıldı" demek değildir, "ikinci sepette 2 numaralı ürün var" demektir.

Bu matrisin en büyük elemanı, en çok birlikte satılan ürün çiftini verir. Burada en büyük değer 2'dir: ikinci ürünle en çok satılan ürün dördüncü üründür. En büyük değer birden fazla yerde çıkabilir. Hepsini bulmak analiz için fayda sağlar.

Hoca sonra veriyle biraz oynadı. Üçüncü sepete 4 numaralı ürünü koymak bu sonucu değiştirmedi, çünkü üçüncü sepette 2 yoktu ve biz 2 ile 4'ün birlikteliğine bakıyorduk. Sonra üçüncü sepette 2 olsun dedi ve kayıt sayısını $n = 7$ yaptı. Yukarıdaki programın çıktısı şudur:

```
Sepet-urun matrisi A:
  0  0  0  0  0
  0  0  1  0  1
  0  1  1  0  1
  0  0  1  1  0
Urunlerin birlikte satis sayilari T:
  0  0  0  0  0
  0  0  1  0  1
  0  0  0  1  2
  0  0  0  0  0
  0  0  0  0  0
```

Burada $T[2][3] = 1$ eklendi, en büyük değer yine $T[2][4] = 2$'dir.

2.4 if'ten Kurtulmak

Slaytta `if`'i kaldırmanın yolu soruluyordu. A matrisi yalnızca 0 ve 1 içerdiği için iki değeri karşılaştırmak yerine çarpabiliriz. İkisi de 1 ise sonuç 1, biri 0 ise sonuç 0 olur. Ürünler o sepette ortak değilse sayaca hiçbir şey eklenmez:

```
T[i][j] += A[k][i] * A[k][j];
```

Hoca bunu `T[i][j] = T[i][j] + ...` diye yazdı ve `+=` ile de yazılabileceğini söyledi. Hocanın gerekçesi şudur: Karşılaştırma da aslında arka planda bir çıkarma işlemidir, burada onun yerine çarpma kullanılır. Program çalıştırıldığında aynı sonuç elde edildi.

2.5 Üçlü Ürünler ve Üç Boyutlu Matris

Hocanın öğrencilere bıraktığı soru şuydu: En çok birlikte satılan ikililer değil de üçlüler istenseydi ne yapardık? O zaman matris yetmez, üç boyutlu bir matris yani küp gerekir: $T[i][j][k]$, birinci, ikinci ve üçüncü ürünün birlikte satılma sayısını tutar. Üç boyutlu yapı üç değişkenin ilişkisini gösterir. Normal bir matris satır sayısı kadar dizi içeriyorsa, burada k tane matrisin üst üste durduğunu düşünürüz. Hoca küp çizimini teknik resim derslerinde çizdikleri şekillere benzetti. Dörtlü ürünlere çıkınca iş daha da karışır. Üçlü ürünlerin kodu pazartesi yazılacak.

2.6 Soru: Sorularla Kişi Bulan Oyunlar

Soru: Bir sesli asistanda aklınızdan bir kişi veya nesne tutuyorsunuz (örneğin kağıt). Sistem "şu var mı, bu mu?" türünden sorular soruyor ve verdiğiniz cevaplara göre tuttuğunuz şeyi buluyor. Bu da benzer bir mantıkla mı çalışıyor? **Cevap:** Benzer bir mantıkla yapılmış olabilir. Bu aslında işbirlikçi filtreleme (collaborative filtering) ve öneri sistemleri (recommendation system) konusudur. Temelleri burada anlatılanlardır.

2.7 Karmaşıklık

A matrisini oluşturan döngü n kez çalışır, burada sorun yoktur. Asıl yük T'yi oluşturan kısımdadır. Ürün sayısına n diyelim. Hoca önce $\frac{n(n+1)}{2}$ dedi ve hemen düzeltti: j hep $i+1$'den başladığı için iç içe ürün döngüleri

$$\frac{n(n-1)}{2}$$

kez döner, bu da n^2 mertebesindedir. Her biri için bütün sepetler gezilir. Toplam işlem sepet sayısı $\times\, n^2$ olur. Sepet sayısını yaklaşık n alırsak n^3'e giden bir algoritma elde ederiz. Temel işlem `if`'teki karşılaştırma da olsa, çarpma da olsa bu sayı değişmez. Ürün ve sepet sayısı çok büyürse algoritma zorlanır.

Bellek açısından da başlangıç listesine ek olarak A için ürün sayısı $\times$ sepet sayısı ($m \times n$ veya yaklaşık n^2), T için ürün sayısı $\times$ ürün sayısı (n^2) kadar yer gerekir. Yani bu yöntem iki tane n^2'lik ek yer ister. Daha iyi teknikleri veri madenciliği dersinde anlatılır.

3. Makrolar

3.1 Makro Nedir?

Makro bir fonksiyon gibi düşünülebilir ama tamamen fonksiyon değildir. İlk derslerde derleme aşamalarını görürken `#define`'ın bir yerine koyma (substitution) yaptığını görmüştük. `#define` yalnızca sabit tanımlamak için değildir, fonksiyon benzeri bir yapı da kurabilir. Bu yapı da sonuçta yerine koymadır. Şöyle bir örnek yazıldı (`sqr` bir kütüphanede bulunabilir diye adı `MYSQR` seçildi):

```
#define MYSQR(x) x*x

int y = MYSQR(5);
```

Derleme aşamasında `MYSQR(5)` görülen yere `5*5` yazılır, tanımdaki ifade kopyalanır. Program çalıştığında y 25 olur.

3.2 Fonksiyondan Farkı

Bir fonksiyon çağrıldığında fonksiyonun tanımlandığı yere gidilir, sonra kalınan yere dönülür. Hoca bunu satır numaralarıyla izletti: 8. satırdan 11. satıra git, 13. satırdan tekrar 8. satıra dön, 9. satırdan devam et. Bu sırada kalınan yer gibi bilgiler yığına (stack) yedeklenir. Makroda ise böyle bir şey yoktur. Hocanın benzetmesiyle:

> **Hocanın vurgusu:** Biz fonksiyona gideriz, makro bize gelir. Makronun en büyük sıkıntısı fonksiyon gibi görünmesidir, oysa fonksiyon değildir.

Makro bize geldiği için yedekleme olmaz, kod yalnızca yerine koyma nedeniyle biraz genişler ve kısmen daha hızlı çalışır.

3.3 Parantez Sorunu

Makronun bir yan etkisi vardır. `MYSQR(5-3)` kaç verir? Akla ilk gelen 4'tür, ama 4 gelmez. Yerine koyma metin üzerinde yapıldığı için x yerine `5-3` gider:

$$5 - 3 \times 5 - 3$$

Önce çarpma yapılır:

- $3 \times 5 = 15$, ifade $5 - 15 - 3$ olur.
- $5 - 15 = -10$.
- $-10 - 3 = -13$.

Hoca aynı hesabı $-15-3 = -18$, $-18+5 = -13$ sırasıyla yaptı. Sonuç -13'tür. Doğru sonucu almanın iki yolu vardır:

- **Çağırırken parantez:** `MYSQR((5-3))` yazılırsa ifade $(5-3) \times (5-3) = 4$ olur.
- **Tanımda parantez:** Her parametre kendi içinde, bütün ifade de dışarıdan parantez içine alınır.

```
#define MYSQR1(x) ((x)*(x))
```

Derste çıktılar sırasıyla 25, -13, 4 ve 4 oldu.

> **Hocanın vurgusu:** Asıl yapılması gereken `MYSQR1`'deki gibidir. Gelen her parametre tanımda parantez içine alınır. Çağıran taraftan parantezle kurtarmak çok doğru değildir.

3.4 Birden Fazla Parametre

Makro bir parametreyle sınırlı değildir. İki sayıdan küçüğünü veren makro, koşullu operatörle yazılır:

```
#define MIN(x,y) ((x)<(y)?(x):(y))
```

Tanımda eşittir kullanılmaz, araya boşluk bırakılır. Sonunda noktalı virgül de kullanılmaz. `MIN(3,5)` 3 verir.

Soru: Makro her değerle çalışır mı? (Soru altyazıda duyulmuyor, cevaptan anlaşılıyor.) **Cevap:** Burada bir değişken tanımı yapılmadığı için her değerle çalışır. Ama gönderdiğiniz şeyin uygun olması gerekir. Örneğin `"abc"` gönderirseniz derleme zamanı hatası alırsınız, çünkü giden şey işaretçi gibi bir dizidir.

Derste bir de `MIN(3, 3.6)` denendi. Sonuç anlamsız bir sayı çıktı. Hoca bunun tip dönüşümüyle ilgili olabileceğini, `%d` yerine `%f` gibi bir biçim gerektiğini söyledi.

> **Ders sonrası not:** Sebep doğrudur. `MIN(3, 3.6)` ifadesinin tipi `double`'dır ve `double` bir değeri `%d` ile yazdırmak hatalıdır. `gcc -Wall` bu satırda `-Wformat` uyarısı verdi ve anlamsız bir tam sayı yazdı, `%f` ile sonuç `3.000000` oldu. Derste hoca bakmayı sonraya bıraktığı için bu sonuç derste görülmedi. `"abc"` için de derleme hatası `gcc` ile doğrulandı.

3.5 Ne Zaman Makro, Ne Zaman Fonksiyon?

Tek satırlık, kolay ve kısa işlerde makro hız açısından fayda sağlar, en azından bizi yavaşlatmaz. Ama birçok satırdan oluşan işler için, örneğin bir matrisi ekrana yazmak için uzun uzun `define` yazmak makul değildir, fonksiyon yazmak daha iyidir. Makroların ayrıntısı gelecek dönem Yapısal Programlama dersinde görülecek.

4. Stringler

4.1 String Bir Karakter Dizisidir

String aslında bir karakter dizisidir (array of char). Normal dizilerden iki farkı vardır: Elemanları karakter türündedir ve sonunda otomatik olarak `'\0'` durdurma karakteri bulunur.

> **Hocanın vurgusu:** C'de string diye bir veri tipi yok. Başka dillerde olabilir, burada yok.

```
char name[50] = "deneme";
```

Bu tanım 50 karakterlik bir yer ayırır. Yerin ilk 6 karakterine `deneme`, 7. karakterine `'\0'` yazılır. Stringimiz aslında 6 karakterlidir. `string.h` içindeki `strlen` fonksiyonu da 6 verir. Bu başlık dosyasında string diye bir tip yoktur, stringlerle ilgili temel fonksiyonlar vardır.

4.2 Uzunluğu Kendimiz Bulalım

String bir dizi olduğu için klasik bir tam sayı dizisi gibi kullanılabilir. Bellekte `d`, `e`, `n`, `e`, `m`, `e`, `\0` diye saklanır. Ne zaman biteceğini bilmediğimiz için `while` kullanılır. Hoca önce kısa yazımı gösterdi, sonra sonucu karşılaştırdı (`gcc -Wall` ile uyarısız derlendi, çıktısı 6, 7, 6):

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char name[50] = "deneme";
    int i = 0;

    printf("%d\n", (int) strlen(name));

    while (name[i++]);
    printf("%d\n", i);

    i = 0;
    while (name[i] != '\0')
        i++;
    printf("%d\n", i);
```

```
    return 0;
}
```

`while (name[i++]);` döngüsünün içi boş gibi görünür ama aslında `i++` ile doludur. Önce `name[i]`'ye bakılır, sonra `i` artırılır. `name[i]` `'\0'` olduğu anda döngü biter, ama `i` o anda da bir kez artmıştır. Bu yüzden sonuç 7 çıkar: `'\0'` de sayılmış olur. Bu yazımda uzunluk `i - 1`'dir. Artırma döngünün içine yazılırsa sonuç 6 olur.

Hoca, bazı öğrencilerin yaptığı bir hataya da dikkat çekti: Aynı işi yapan ikinci bir `t` sayacı tutmak. `i` ve `t` ikisi de sıfırdan başlar, başa baş gider. `t` gereksiz bir değişkendir, iş `i` ile halledilir.

4.3 Büyük Harfi Küçüğe Çevirmek

Büyük harflerin ASCII kodları 65'ten, küçük harflerin kodları 97'den başlar. Aradaki fark $97 - 65 = 32$'dir. Yani `name[i]`'ye 32 eklersek büyük harf küçük harfe döner. Karakter arka planda örtük olarak tam sayıya dönüştürülür.

```
name[i] += 32;
printf("%s\n", name);
```

`%s`, dizinin başlangıç adresinden başlayıp `'\0'`'ı gördüğü yerde durarak yazar. 50 karakterin hepsini yazmaz.

Derste ilk denemede hiçbir şey değişmedi. Bir öğrenci sohbetten `while` içinde `i`'nin artmadığını yazdı. Hoca asıl sebebin bir önceki döngü olduğunu gördü: `i` önceki döngüde stringin sonuna kadar gelmişti, sıfırlanmadığı için yeni döngüye hiç girilmedi. `i = 0` yazılınca bu sefer `i` artmadığından sürekli sıfırıncı elemana 32 eklendi ve döngüden çıkılamadı. `i++` eklenince bütün karakterler değişti.

Sonuç hocanın gösterdiği gibi şöyleydi: Önce `D`, `d` oldu. Sonra `d`'ye bir daha 32 eklenince başka bir karaktere ulaşıldı (ara adım eklendi: $100 + 32 = 132$). Diğer harfler de 32 eklenince değişti. Buradan çıkan ders, stringlerin dizilerden farksız olduğu ve istediğimiz elemana doğrudan erişebildiğimizdir.

> **Ders sonrası not:** Bu döngü küçük harflere de 32 eklediği için bütün metni düzgün küçültmez (`gcc` ile `Deneme` üzerinde denendi, küçük harfler anlamsız karakterlere dönüştü). Yalnız büyük harfleri çevirmek için `if (name[i] >= 'A' && name[i] <= 'Z')` denetimi gerekir.

5. Şifreleme: Sezar, Afin, Tablo ve Harf Sıklığı

Stringler üzerindeki uygulamamız şifrelemedir. Bir şifre girdiğiniz yerde onu açık metin olarak saklamak istemezsiniz.

5.1 Sezar Şifreleme

En basit yöntemlerden biri olan Sezar şifrelemede her karaktere sabit bir sayı eklenir:

$$y = x + c$$

Burada c, 3 veya 5 gibi bir sabittir. Yeni oluşan string şifreli stringdir. Hoca $c = 3$ ve `abcd` için kodu yazdı. Orijinal metin korunmak istendiği için şifreli metin ayrı bir diziye yazılır. Çözme ise fonksiyonun tersidir: $x + c$'nin tersi $x - c$'dir. Bütün program şudur (`gcc -Wall` ile uyarısız derlendi):

```
#include <stdio.h>

int main(void)
{
    char name[50] = "abcd";
    char name2[50] = "";
    char name3[50] = "";
    int c = 3;
```

```
    int i = 0;

    while (name[i] != '\0') {
        name2[i] = name[i] + c;
        i++;
    }
    name2[i] = '\0';

    i = 0;
    while (name2[i] != '\0') {
        name3[i] = name2[i] - c;
        i++;
    }
    name3[i] = '\0';

    printf("%s\n%s\n%s\n", name, name2, name3);
    return 0;
}
```

Çıktı adım adım şöyledir:

- Orijinal metin: `abcd`.
- Şifreli metin: `a + 3 = d`, `b + 3 = e`, `c + 3 = f`, `d + 3 = g`, yani `defg`.
- Çözülmüş metin: `defg`'nin her karakterinden 3 çıkarılır, yine `abcd` olur.

Hoca burada bir `while` içinde `i++` yazmayı yine unuttu, sonra ekledi. Ayrıca çözmeden önce `i`'yi sıfırladı.

Önemli bir nokta da sonlandırma karakteridir. Döngü `name`'in `'\0'`'ını `name2`'ye aktarmaz. Bu nedenle döngüden sonra `name2[i] = '\0';` yazılır. Döngüden çıkıldığında `i` tam son karakterin bir sonrasındadır.

> **Hocanın vurgusu:** `'\0'` stringin sonuna gelindiğini garanti eden karakterdir. Yazmazsanız bazı durumlarda çöp değerler görebilirsiniz.

Hoca bu çöp değerleri göstermek için `'\0'` satırını silip derledi, bir değeri 40 yaparak da denedi ama göremedi. "Şu an şanssızız" dedi.

> **Ders sonrası not:** Çöp değerin görülmemesinin sebebi büyük olasılıkla `name2`'nin `char name2[50] = "";` ile tanımlanmış olmasıdır. Kısmi ilk değer verilen bir dizide kalan elemanlar sıfırla dolar. Yani `name2`'nin her yeri zaten `'\0'`'dır. İlk değer verilmemiş yerel bir dizide aynı kodla çöp karakterler büyük olasılıkla görülebilir.
>
> **Ek bilgi (derste anlatılmadı):** Klasik Sezar şifrelemede alfabe büyük olasılıkla döngüsel kabul edilir. Dersteki `x + c` biçiminde `z`'ye 3 eklenince alfabenin dışına çıkılır (`{` karakteri). Döngüsel biçim `(x - 'a' + c) % 26 + 'a'`, çözme biçimi `(y - 'a' - c + 26) % 26 + 'a'`'dır.

5.2 Afin Şifreleme

Biraz daha karmaşık bir yöntem afin şifrelemedir:

$$y = ax + b$$

Bu kez iki sabit vardır, örneğin $a = 2$, $b = 3$. Hoca, tersinin "$\frac{b}{a}$" olduğunu, ama tersin her zaman çalışmadığını söyledi. Şifreleme her zaman çalışır, deşifreleme her zaman çalışmaz. Nedeni pazartesi konuşulacak.

> **Ders sonrası not:** Hoca tersi $\frac{b}{a}$ olarak söyledi. $y = ax + b$'nin tersi ise $x = \frac{y-b}{a}$'dır. Harf alfabesi üzerinde $y = (ax + b) \bmod 26$ alındığında bölme yerine a'nın 26'ya göre tersi gerekir. Bu ters yalnızca $\gcd(a, 26) = 1$ ise vardır. Dersteki $a = 2$, $b = 3$ değerlerinde `a` ($x = 0$) ve `n` ($x = 13$) aynı değere, 3'e gider (python ile doğrulandı).

Yani geri dönüş tek değildir. Hocanın pazartesiye bıraktığı neden büyük olasılıkla budur.

5.3 Tabloyla Şifreleme

Bir diğer yöntemde olası bütün karakterler (büyük harfler, küçük harfler ve ASCII kodundaki her şey) bir matrise yerleştirilir. Yerleştirme rastgele olabilir veya sıralı dizilip sonra karıştırılabilir.

Sonra metnin her karakteri matriste aranır ve bulunduğu satır ile sütun numarasından bir sayı üretilir. Slayttaki örnekte `A`'nın karşılığı 1'e 1, yani 11; `B`'nin karşılığı 2'ye 2, yani 22'dir. Slayt indisleri 1'den başlatıyordu. Sonuçta bir tam sayı dizisi elde edilir. Çözerken sayının ilk basamağı satır, sonraki basamağı sütun olarak okunur ve matristen harf alınır. Hoca kodu zor olmadığını söyleyerek öğrencilere bıraktı: Matrisi doldurun, stringi karakter karakter gezin, her karakterin matristeki i ve j indisini bulup yazdırın.

> **Ders sonrası not:** Hoca 256 karakterlik ASCII tablosu için "en az 8'e 8'lik bir matris" dedi. $8 \times 8 = 64$ hücredir. 256 karakter için en az 16×16 gerekir. Standart ASCII'nin büyük olasılıkla 128 karakter olduğu, 256'ya genişletilmiş kod sayfalarıyla çıkıldığı da unutulmamalıdır (çarpımlar python ile doğrulandı). Ayrıca çarpı 10 kodlaması büyük olasılıkla yalnızca satır ve sütun numaraları tek basamaklıysa geri çözülebilir.

5.4 Harf Sıklığıyla Şifre Kırmak

Tablo elimizde olmasaydı ne yapardık? Her harf hep aynı sayıya dönüştüğü için, örneğin her `A` hep 11 olduğu için, şifreli metinde en çok tekrar eden sayı o dilde en çok kullanılan harfe karşılık gelebilir. Hoca derste öğrencilere Türkçede en sık harfi sordu. Hoca önce E dedi, sonra bir öğrencinin hatırlatmasıyla düzeltti: Türkçede en sık harf A, İngilizcede E'dir. Türkçede sıra A, E, İ, N, R, L olarak söylendi. Bu yalnızca bir tahmindir, kesin sonuç vermez. Bu konuda olasılıksal şifreleme yöntemleri ayrıca çalışılır.

Bu saldırıyı zorlaştırmanın yolu da slaytta verildi. M matrisinde her karakter, o dildeki sıklığıyla orantılı sayıda yer alır. Örneğin Türkçede matriste bir sürü `A` olur. Şifrelerken bütün `A`'ların kodları bulunur ve aralarından biri rastgele seçilir. Böylece bir harf hep aynı sayıya dönüşmez.

Hoca şifreleme kısmının zaten bildiğimiz dizi işlemlerinden ibaret olduğunu, bu yüzden üzerinde çok durmaya gerek olmadığını söyledi.

6. Metinde Birlikte Geçen Kelimeler

6.1 Arka Arkaya Gelen Kelimeler

Bu problem aslında sepet örneğinin farklı bir uygulamasıdır. Hocanın deyişiyle sepetimiz cümledir: Her cümle bir sürü kelime içerir ve kelimeler arasındaki ilişkilere bakarız. Metin, kelime ID'lerinden oluşan n elemanlı bir M dizisi olarak tutulur. W ise farklı kelime sayısı $\times$ farklı kelime sayısı boyutunda bir matristir. Önce sıfırlanır. Sonra metin baştan sona gezilerek arka arkaya gelen kelimeler sayılır:

```
W'yi sıfırla
i = 1'den n'ye kadar:
    W[M[i]][M[i+1]] = W[M[i]][M[i+1]] + 1
```

Bu matristen hangi kelimenin hangi kelimeyle peş peşe geldiğinin sıklığı okunur.

> **Ders sonrası not:** Hoca döngüyü metnin boyutuna kadar anlattı. $M[i+1]$ kullanıldığı için indisler 1'den başlıyorsa son i büyük olasılıkla $n-1$ olmalıdır, yoksa dizinin dışına çıkılır.

6.2 Bir Kelimenin Çevresindeki Kelimeler

İkinci soru şudur: Bir kelime verildiğinde onun k kadar ötesinde ve k kadar gerisinde en çok hangi kelimeler vardır? Yöntem aynıdır, W sıfırlanır, ama bu kez bulunduğumuz yerden k kadar

ileri ve geri sayılır. Dışarı taşmamak için döngü k kadar ileriden başlar ve n'den k kadar geride biter. Her kelimenin çevresinde en çok hangi kelimelerin bulunduğu böyle elde edilir. Hoca bu sözde kodun değişkenlerini bir cümle üzerinde kendinizin izlemenizi önerdi. C'de bu işin hemen yazılamayacağını, ek kütüphaneler gerektiğini de ekledi.

6.3 Uygulama: Deasciification

Bu yöntem doğal dil işleme ve metin madenciliğinde kullanılır. Bir uygulaması deasciification'dır. Türkçe karakterleri bozulmuş bir metinde hangi kelimenin kastedildiğine, birlikte geçtiği kelimelerin sıklığına bakılarak karar verilir.

Hocanın ilk örneği toprakla ilgiliydi: "Toprak" kelimesinin yanında "kıllı" kelimesinin geçmesi neredeyse hiç görülmez. O halde metin "kıllı toprak" diye bozulmuşsa doğru olan seçilir ve anlam düzelir. Sonra öğrencilerden örnek istedi. "Kurt ormanda / Kürt ormanda" önerisinde iki biçim de geçerli isim olduğu için sonuç çıkmayabileceğini söyledi. Daha iyi örnek "Şişli Caddesi"dir. ş harfleri s'ye dönüşürse "Sisli Caddesi" olur. Aynı şekilde "Şişli Mecidiyeköy" de "Sisli Mecidiyeköy" olur. Hoca, Mecidiyeköy'ün o gün sisli olabileceğini de şakayla ekledi. Yani yöntem kesin değildir.

Bu her dilde olabilir. Örneğin Almancadaki noktalı a harfi normal a'ya dönüşünce farklı bir anlam çıkabilir. Hoca, PDF'ten Word'e dönüştürürken bazı ı ve i harflerinin tanınmadığını, eski Word sürümlerinde karakterlerin bozulduğunu, Dev-C++'ta da Türkçe karakterlerin başka karakterlere dönüştüğünü gösterdi.

Soru: Bu, başka alfabelerde de yapılabilir mi? (Soru altyazıda duyulmuyor, cevaptan anlaşılıyor.) **Cevap:** İstediğiniz alfabede yapabilirsiniz. Kiril alfabesinde de bir karaktere en çok benzeyen başka bir karakter vardır. Bizde ı ve i gibi: i, ı'ya dönüşmüşse ve anlam bütünlüğü oluşmuyorsa aynı mantıkla geri çevrilebilir.

6.4 Anlamsal Yakınlık

Slaytta bir fikir daha vardı: Birlikte geçen kelimeler anlamca birbirine yakındır. Word2Vec, FastText ve GloVe gibi araçlar bu fikirle üretilmiştir. Böylece yapıca benzemeyen ama anlamca benzeyen kelimeler bulunabilir. Hocanın örneği şudur: "Cimbom kazandı" ile "Galatasaray yendi" aynı anlama gelir, oysa "Cimbom" ile "Galatasaray" arasında harf açısından bir benzerlik yoktur. Kullanım sıklıkları analiz edilerek birinin diğerine belki %80, belki %90 yakın bir anlam taşıdığı hesaplanabilir.

7. Görüntüde Gürültü Temizleme: Ortalama Filtresi

7.1 Problem: Tuz Biber Gürültüsü

Hoca görüntü işlemede çok kullanılan bir resmi (Lena) gösterip "bir bozukluk hissediyor musunuz?" diye sordu. Öğrenciler karıncalanma ve gürültü dedi. Bu gürültüye tuz biber gürültüsü (salt and pepper noise) denir: Beyaz noktalar tuz, siyah noktalar biberdir. Amacımız bu gürültüyü temizlemektir.

Resim açılıp okunduğunda elimizdeki veri bir matristir. Bu matris resmin piksellerini tutar. Düzeltilen matris yeni bir dosyaya kaydedilir. C'de resim okumak kütüphanesiz yapılan bir iş değildir ve bu dersin konusu değildir. Bu yüzden okuma ve yazma hazır kodla yapıldı.

Örnekte resim gri seviyeli ve 8 bitliktir, yani 256 farklı değer alır. Beyaz 255 (veya 0-1 aralığında 1), siyah 0'dır. Normal bir resimde pikseller arasında bir uyum (harmoni) vardır: Değerler 80, 85, 85, 90 diye giderken araya bir anda beyaz veya siyah bir nokta girerse, işte bu gürültüdür.

Hoca bir PPM dosyasını metin olarak açtı. İçinde beyaz bölgeleri gösteren 255'ler görülüyordu. Dosyanın kopyasında birkaç 255'i sıfır yapıp resmi yeniden açınca bir köşede bozulma görüldü. Resim büyük, değişiklik küçük olduğu için fark zor görülüyordu.

7.2 Fikir: Pencere Ortalaması

Kelimelerde anlamsal yakınlığa baktığımız gibi, görüntüde de bir noktanın çevresine bakabiliriz. Pikselin 3×3'lük çevresinin ortalamasını alırsak, gürültülü bir nokta yerine çevresindeki renge yakın bir değer gelir. Hoca sayısal örnek olarak 85'ler arasındaki 1 değerini aldı: $85 + 1 + 85 + \ldots$ toplamı 9'a bölünür ve sonuç "belki 56 gibi" bir değer olur dedi.

> **Ders sonrası not:** Hocanın tahmini 56 ve "50'ler, 60'lar"dır. Sekiz komşu 85, merkez 1 ise toplam $8 \times 85 + 1 = 681$, ortalama $\frac{681}{9} \approx 75{,}67$'dir. Merkez 0 ise $\frac{680}{9} \approx 75{,}56$, merkez 255 ise $\frac{935}{9} \approx 103{,}89$ çıkar (python ile doğrulandı). Hocanın asıl noktası değişmez: Sonuç 85'e tam dönmez.

Gürültünün nerede olduğunu bilmiyoruz. O yüzden bütün resim baştan sona işlenir. Filtre bir pencere gibi gezdirilir:

1. 3×3'lük alanın ortalaması hesaplanır ve ortadaki piksele yazılır.
2. Pencere bir kaydırılır, aynı işlem yapılır.
3. Satır bitince pencere aşağıya kaydırılır.
4. Bütün matris gezilene kadar devam edilir.

Bu yöntemde resmin kenarlarına dokunulamaz, dokunmanın başka yolları vardır ama burada ilgilenmiyoruz. Filtre boyutu 3×3, 5×5, 7×7, 9×9, 11×11 olabilir. Genelde tek sayılar kullanılır. Matrisi gezmek için iki, filtre penceresini gezmek için iki `for` gerekir.

7.3 Döngü Sınırları ve Kod

Sıfır noktasından başlamakta sorun yoktur, çünkü filtre ileriye doğru işlem yapar. Resmin sonuna kadar gidemeyiz: Pencere dışarı taşar. Bu yüzden döngü sınırı "sütun sayısı + 1 - filtre boyutu" olur. Bir fazlasını almamızın sebebi, üst sınırın döngüye dahil olmamasıdır. Fonksiyon şöyledir (bir denemeyle `gcc -Wall` altında uyarısız derlendi; dersteki değerlerle merkez 1 iken sonuç 75 çıktı):

```
#define MAX 512

void meanFilter(int resim[MAX][MAX], int B[MAX][MAX], int M, int N)
{
    int i, j, m, n;
    int toplam = 0;
    int filtre = 3;

    for (i = 0; i < N + 1 - filtre; i++) {
        for (j = 0; j < M + 1 - filtre; j++) {
            for (m = 0; m < filtre; m++)
                for (n = 0; n < filtre; n++)
                    toplam += resim[j + m][i + n];
            B[j][i] = (float) toplam / (filtre * filtre);
            toplam = 0;
        }
    }
}
```

Dıştaki iki döngü resmi, içteki iki döngü pencereyi gezer. `filtre * filtre`, filtre 3 ise 9'dur; böyle yazınca kod parametrik olur. Her pencereden sonra `toplam` sıfırlanmalıdır, derste bu satır hemen hatırlandı. Hoca `i` ve `j`'nin yer değiştirdiğini, kurgunun ona göre olduğunu söyledi. Sonuç `B` matrisinde oluşur ve dosyaya yazılır.

Dosya adı için `char yeni = "...";` yazılınca derleme hatası alındı. String ancak bir karakter dizisine atanabilir: `char yeni[] = "resimYeni.pgm";`. Yazma fonksiyonuna `B` matrisi, boyutlar ve `q` (kuantizasyon seviyesi) gönderilir.

> **Ders sonrası not:** Hoca, sonuç sıfır gelebileceği için önce `float`'a dönüştürdüğünü söyledi. Oysa `B` bir tam sayı matrisi olduğu için sonuç yine tam sayıya kırpılır. Bu nedenle `float` dönüşümü burada sonucu büyük olasılıkla değiştirmez: Dersteki değerlerle $681/9$ iki yoldan da 75 verir (python ile doğrulandı).

> **Ders sonrası not:** Hoca sonucu pencerenin ortasına yazacağımızı anlattı, ama kodda sonuç `B[j][i]`'ye, yani pencerenin sol üst köşesine yazılıyor. Bu yüzden resim büyük olasılıkla yarım pencere kadar kayar ve son satır ile sütunlarda yeni değer oluşmaz. Ortaya yazmak için pencereyi `i` ve `j` çevresinde gezdirmek gerekir.

7.4 Sonuçlar ve Filtre Boyutu

Program çalıştırılınca eski ve yeni resim karşılaştırıldı. 3×3 filtre gürültüyü kısmen azalttı ama tamamen yok etmedi. Kenardaki bozukluk için ayrıca çalışmak gerekir.

> **Hocanın vurgusu:** Ortalama filtresi başarılı bir filtre değildir. Ortalama hesabında gürültüyü de işin içine katıyoruz.

Hoca filtre boyutunu 9 yapınca ne olacağını sordu. 81 değer işin içine girdiği için resim daha bulanık olur. Gerçekten de gürültüler neredeyse yok oldu ama resim iyice bulanıklaştı. Filtre 21 yapılınca hem resim iyice kayboldu hem işlem daha geç bitti. Çünkü işlem sayısı $M \times N \times$ filtre $\times$ filtre kadardır.

> **Ders sonrası not:** Hocanın 21 için söylediği sayı altyazıda "21 çarpı 400" diye geçiyor. $21 \times 21 = 441$'dir. Yani 3×3'e göre $\frac{441}{9} = 49$ kat fazla işlem vardır (python ile doğrulandı).

Demek ki filtre boyutunu büyütmek çözüm değildir, başka bir filtre gerekir. Hoca, Photoshop'ta tuz biber gürültüsünü gideren hazır bir komut olduğunu hatırlatıp bunun nasıl yapıldığını pazartesiye bıraktı.

> **Ek bilgi (derste anlatılmadı):** Tuz biber gürültüsünde büyük olasılıkla pencerenin ortalaması yerine ortancası (median filter) kullanılır. Uç değerler ortanca değeri pek etkilemez, bu yüzden hem gürültü ortalamaya karışmaz hem de kenarlar daha az bulanıklaşır.

Soru: 3×3 içindeki değerlerin standart sapmasının dışında kalan değerleri ortalamaya dahil etmesek olur mu? **Cevap:** Standart sapmayı hesaplayınca ortalamaya yakın bir değer bulunur, bunu deneyebilirsin. Çözüm olabilir, daha başka çözümler de var.

Hoca ayrıca resimdeki kenarları (şapkanın çizgileri, göz, burun) bulma problemini de sonraki bir soru olarak andı.

8. Düzgün Bir Rapor Nasıl Hazırlanır?

Hoca ders sonunda Algoritma Analizi dersinde bir öğrencinin hazırladığı örnek bir raporu gösterdi. Bu kesin bir format değildir. Size hazır bir şablon verilmemişse herkes kendine özgün bir rapor hazırlayabilir. Ama her raporda az çok bulunması gerekenler vardır:

- **Kapak:** Ad, soyad, numara, iletişim bilgisi (en çok kullandığınız e-posta), ders ve grup, hangi ödev olduğu, yürütücü, tarih.
- **İçindekiler:** Word'de kolayca oluşturulur.
- **Bölümler:** Kısa bir giriş, yöntemler, ekran görüntüleri, önerileriniz. Adım adım anlatımlar maddeler halinde yazılır.
- **Hizalama:** İki yana yaslı yazı okunabilirliği artırır. Akademik metinlerde böyledir.
- **Denklem ve şekiller:** Denklemler açıklanır. Şekil yazısı şeklin altına, tablo yazısı tablonun üstüne yazılır. Elle yazdıklarınızın fotoğrafını düzgün çekin.
- **Kod:** İstenmişse program kodu renkli eklenirse daha güzel olur.

Metni yazdıktan sonra yarım saat uğraşmak, bu formatı oluşturmak için yeterlidir.

9. Çok Büyük Sayılarla Çalışan Algoritmaları Denemek

Soru: Diffie-Hellman anahtar değişimi algoritmasında taraflar anahtarları takas etmek için sürekli kuvvet alıyor. C'de en büyük değer 64 bit. Rastgele alınan değerler taşınca, mod alındığında iki tarafta oluşan anahtarlar farklı olabiliyor. Ne yapılmalı? **Cevap:** Küçük verilerle test edin. Rastgele sayıyı daha düşük bir aralıkta üretin. Amaç algoritmanın çalışma mantığını göstermekse gerçek dünyadaki çok büyük sayılara girmeye gerek yok. RSA'da da benzer bir

mantık vardır: Çok büyük asal sayıların çok büyük kuvvetleri hesaplanır. Elinizdeki bilgisayarların çoğu buna müsait değildir. Bu algoritmaların kaynaklarında da örnekler küçük sayılarla gösterilir. 3, 5, 7, 11 gibi sayılarla çalışmak yeterlidir.

> **Ek bilgi (derste anlatılmadı):** Kuvvet alırken taşma büyük olasılıkla, sonucu en sonda değil her çarpımdan sonra mod alarak büyük ölçüde önlenir. Bu yönteme modüler üs alma denir.

Dersin Toparlaması

Bu derste aynı fikrin farklı yüzlerini gördük. Sepet analizinde karışık bir listeden önce 0-1 matrisi, sonra ürün-ürün birlikte satış matrisi kuruldu. Çarpma ile `if`'ten kurtulundu ve algoritmanın n^3 işlem, iki n^2 bellek gerektirdiği görüldü. Birlikte geçen kelimeler aynı sayma mantığının metne uygulanmasıdır. Makrolar, yerine koyma mantığı ve parantezin önemiyle fonksiyonlardan ayrıldı. Stringler ise sonunda `'\0'` olan karakter dizileri olarak işlendi ve bu bilgiyle Sezar şifrelemesi kodlandı. Son olarak görüntü bir matris olarak ele alındı, ortalama filtresiyle gürültü azaltıldı ama filtrenin gürültüyü de ortalamaya kattığı için yetersiz kaldığı görüldü.

Pazartesi günü en çok birlikte satılan üçlü ürünlerin kodu, afin şifrelemede çözmenin neden her zaman çalışmadığı ve tuz biber gürültüsü için daha iyi bir filtre konuşulacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.